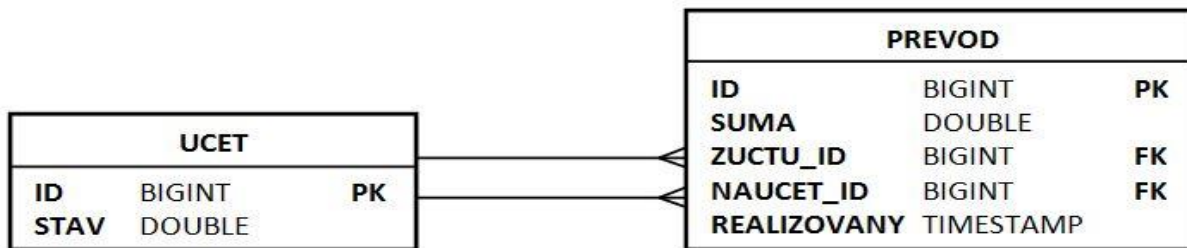


Implementačná úloha 1.

Na obrázku je znázornený diagram zjednodušenej databázy banky s dvomi tabuľkami.

- Tabuľka **UCET** obsahuje informácie o účtoch: číslo účtu **ID** je tiež primárny kľúč a **STAV** udáva aktuálnu sumu peňazí na účte.
- Tabuľka **PREVOD** obsahuje informácie o prevodoch peňazí medzi účtami:
 - **SUMA** je prevádzaná suma,
 - **ZUCTU_ID** číslo účtu, z ktorého sa suma prevádza
 - **NAUCET_ID** číslo účtu,, na ktorý sa suma prevádza.
 - V stĺpci **REALIZOVANY** je čas (Timestamp) realizácie prevodu.
Ak prevod ešte nebol zrealizovaný nie je v ňom žiadna hodnota.
 - Kľúč **ID** v tejto tabuľke je autogenerovaný.



Adresár **dbapp** obsahuje maven project pre vývoj aplikácie pracujúcej s touto databázou.

Projekt obsahuje balík **banka**, v ktorom budete vytvárať všetky entitné triedy pre prácu s databázou.

Balík už obsahuje súbor **Banka.java** s deklaráciami dvoch metód:

```
public static long zadajPrevod(EntityManager em, double suma,
                                long zUctu, long naUcet ) throws Exception
```

```
public static int realizujPrevody( EntityManager em)
```

Vašou úlohou je implementácia týchto dvoch **metód podľa špecifikácie**, ktorú **nájdete v zdrojovom súbore v komentári** k deklaráciám týchto metód.

Pokyny k implementácii:

Odporúčam postupovať tak, že najprv dokončíte implementáciu entitných tried a z nich si necháte vygenerovať tabuľky. Nie je potrebné generovať JPAControler a ani sa **neodporúča** jeho použitie.

Informácia k JPA-mapovaniu dátových typov:

- BIGINT odpovedá v JPA číselným typom **long, Long**
- DATE, TIME alebo TIMESTAMP odpovedá dátumovému typu **Date**
- Aktuálny čas získate pomocou `new Date()`;

Dôležité je, aby vygenerované tabuľky mali **presne tú istú štruktúru**, ako je uvedená v **diagrame** hore. Názvy tabuliek, stĺpcov a ich typy musia byť zachované. Skontrolujte si predovšetkým štruktúru prepojovacej tabuľky.

*Poznámka: Pre vaše testovanie môžete si implementovať **main** funkciu. Nebude sa hodnotiť.*

*Projekt obsahuje aj súbor **persistence.xml** s konfiguráciou pripojenia na databázu derby/sample. Ak chcete pre otestovanie použiť inú databázu môžete si ho však upraviť a pridať driver do pom.súboru.*

Pokyny pre odovzdanie:

Balík **banka** so všetkými súbormi, ktoré sú v ňom, zozipujte. Zip súbor nazvite **banka.zip**

a odovzdajte do miesta odovzdania v **AISE**.

Žiadam vás, na zipovanie použite len formát **zip** (nie rar či iné archívne formáty). Zipujte len samotný balík **banka**, nie celý projekt! Je to dôležité pre včasné vyhodnotenie vašich riešení.

Implementačná úloha 2.

V adresári **restapp** nájdete pripravený maven projekt pre vývoj webovej aplikácie s REST servisom. Projekt obsahuje:

- súbor **pom.xml** so všetkými závislosťami potrebnými pre vývoj servisu. Nie je ho potrebné dopĺňať.
- balík **rest**. V tomto balíku budete vytvárať všetky triedy (triedu implementujúcu servis aj dátovú/é triedu)

V balíku **rest** sa už nachádza konfiguračná trieda **REST aplikácie JAXRSConfiguration**. Netreba ju meniť.

Špecifikácia servisu

Servis bude sprístupňovať informácie skúškach zo študijných predmetov. Informácie o skúške z konkrétneho predmetu predstavujú resource, ktorého stav reprezentovaný vo formáte XML má štruktúru:

```
<?xml version="1.0" encoding="UTF-8"?>
<skuska>
  <predmet>VSA</predmet>
  <den>utorok</den>
  <student>Jozef Mrkvicka</student>
  <student>Janko Hrasko</student>
</skuska>
```

kde:

- element **predmet** obsahuje skratku predmetu, ktorá je zároveň jednoznačným identifikátorom skúšky. Teda musí byť zadaná.
- element **den** udáva deň konania skúšky.
- elementy **student** obsahujú mená študentov prihlásených na skúšku.

Poznámka. Uvedený dokument je ukážka. XML dokumenty, s ktorými servis pracuje budú mať elementy s rovnakými názvami a štruktúrou, odlišovať sa budú len textovým obsahom a počtom elementov. Prihlásených študentov môže byť viac ale nemusí byť žiaden. Poradie elementov nie je dôležité.)

Informácie o zmluvách (stav resoursov) budú udržiavané len v pamäti servisu. Vytvorte si pre to vhodné dátové a použite anotáciu **@javax.inject.Singleton**. Nezabudnite tiež na anotáciu **@XmlRootElement**

Servis inicializujte tak, že po štarte (deploymente) bude vytvorený jediný resource, a to pre predmet **VSA**, pričom deň konania skúšky bude **utorok** a prihlásení nebudú žiadni študenti.

Resourisy pre skúšky z ďalších predmetov bude možné pridávať pomocou metódy **POST**.

Koreňový resource nazvite **skuska** a implementujete pre neho metódu:

POST, ktorá slúži pre vytvorenie nového resourcu s informáciami o skúške.

- akceptuje XML dokument (MIME: APPLICATION_XML) podľa horeuvedenej špecifikácie.
- a vracia reťazec (MIME: TEXT_PLAIN) so skratkou predmetu, prevzatou z elementu predmet.

Pozor. Metóda musí najprv preveriť, či resource s danou skratkou predmetu už neexistuje. Ak existuje, neurobí nič a vráti reťazec **NIC**.

Všetky informácie o konkrétnej skúške sú prístupné ako subresource s URI **skuska/{predmet}**, kde reťazec predmet je skratka predmetu. Pre tento resource implementujte nasledujúce metódy:

GET pre MIME TEXT_PLAIN: vráti počet študentov prihlásených na skúšku.

Ak resource neexistuje, malo by sa vrátiť **0** prípadne HTTP 204 alebo HTTP 404 .

GET pre MIME APPLICATION_XML: vyhľadá skúšku podľa skratky predmetu zadaného v URL a vráti informácie ako XML dokument s horeuvedenou štruktúrou. Ak resource pre skúšku z daného predmetu neexistuje, malo by sa vrátiť HTTP 204 alebo 404 (*Pomôcka, implementácia vráti jednoducho null*)

POST slúži na prihlásenie študenta na skúšku. Očakáva reťazec s menom študenta (MIME:

TEXT_PLAIN). Ak je študent s daným menom ešte nie je prihlásený, pridá ho medzi prihlásených študentov. Metóda nevracia nič.

Ak je už študent prihlásený nerobí nič.

Servis umožňuje tiež zistiť skúšky, na ktoré je študent prihlásený. Túto funkcionality implementujte pomocou metódy **GET** pre koreňový resource **skuska**. Meno študenta je zadane ako parameter požiadavky **student** (návod: `@QueryParam("student")`). Metóda zistí, na ktoré skúšky je študent prihlásený a vráti reťazec, zložený zo skratiek predmetov, na ktoré je prihlásený, oddelených medzerou. (MIME: TEXT_PLAIN)

Ak meno študenta nie je zadane ako parameter požiadavky nevráti nič, príp. prázdny reťazec.

Pokyny pre odovzdanie:

Balík **rest** so všetkými súbormi zozipujte. Zip súbor nazvite **rest.zip** a odovzdajte do miesta odovzdania v **AISe**.

Žiadam vás, na zipovanie použite len formát **zip** (*nie rar či iné archívne formáty*). Zipujte len samotný adresár **rest**, nie celý projekt! Je to dôležité pre včasné vyhodnotenie vašich riešení.